SLASH ROLLER: DUELS

Game Design Document — Final Draft (Capstone Cut)

Course: Multi-Agent AI for Game Development (ELVTR)

Assignment: #02 — GDD Final Draft

Author: Juan Diego Lugo

Date: 22 July 2026

Supersedes: Assignment #1 first draft (*Last Call*) — see Revision Note, §8

---

1. Executive Summary

Slash Roller: Duels is a competitive 1v1 arena dueling game: fighting-game intensity in a third-person brawler body. Two fighters enter a small arena with melee combos, a soft-lock targeting system, and one swappable magic slot each. Rounds are short and lethal; the match is **best-of-5**.

- **Win condition:** Take 3 rounds. A round is won by reducing the opponent's health to zero.
- **Loss condition:** The opponent takes 3 rounds first. There are no draws — a 60-second round timer awards the round to the higher-health fighter.

Platform & format: Unreal Engine 5.6, pure native C++ on the Gameplay Ability System (GAS). **Multiplayer over Steam listen server** (host + invite; no dedicated servers at this scope). Solo players fight **bots** — a launch feature, not a stretch goal, so the game is never dead on arrival.

Team & timeline reality: One principal engineer plus a multi-agent AI crew, 5 weeks, on top of an existing shipped combat foundation (GAS core, soft-lock melee, input buffer, Steam sessions — already built and stable).

Steam demo build is the exit criterion.

Elevator pitch: *Nidhogg's* "one more round" loop with *For Honor's* melee weight — built by one engineer and an AI crew in five weeks.

---

2. Game Mechanics (Core Loop)

2.1 The Match Loop

1. **Host or join** — host a listen server and invite via Steam, or start a vs-bot match. Both fighters pick a class and one magic slot.
2. **Fight the round** — melee strikes chain into combos; the soft-lock system keeps strikes tracking the opponent; the magic slot ability is on a 12-second cooldown. First fighter to zero health loses the round (60-second timer breaks stalls in favor of the higher-health

fighter).

3. **Round break (5 seconds)** — scoreboard, announcer line, and one coach

tip; both fighters may swap their magic slot ability.

4. **Repeat to 3 round wins** — match ends, rematch prompt ("one more?").

2.2 Combat Economy

The scarce resource is **commitment**: every attack locks the attacker into recoverable animation frames. Light attacks are fast, low damage, and safe; heavy attacks are slow, high damage, and punishable; the magic slot is a 12-second-cooldown momentum swing (projectile or AoE, by loadout). Blocking negates light attacks but is broken by heavies. This triangle — light beats heavy startup, heavy beats block, block beats light — is the round's entire decision space, executed at speed.

2.3 Why It's a Game and Not a Sandbox

A round cannot be won by spamming one option: each vertex of the triangle is countered by another, cooldown gates the magic swing, and the 60-second timer plus health-lead rule punishes passive play. Bots at three difficulty tiers make the triangle legible to new players before they queue against a human host.

2.4 Modes at Ship

- **1v1 online** (Steam listen server: host + invite)
- **1v1 vs bot** (3 difficulty tiers)
- **Score-attack** — solo gauntlet of escalating bots in the same arenas,

reusing the round flow. This is the "reason to keep it installed" mode for solo players while the PvP population grows.

---

3. Player Experience (What the Player Sees)

- **HUD:** two health bars top-center with round pips beneath (first to 3);

the magic-slot icon bottom-right showing its GAS-driven cooldown sweep; the round timer top-center. Nothing else — the fight is the screen.

- **Soft-lock feedback:** the current `SoftLockTarget` is marked by a subtle

under-feet ring; strikes visibly warp toward it. The player feels "my attacks go where I mean" without managing a lock-on button.

- **Round break screen:** score pips, one **announcer line** ("Round two.

She's limping — finish it."), and one **coach tip** derived from the round just played ("You were parried 4 times — vary your opening string."). A 5-second countdown returns both fighters to spawn.

- **Front end (CommonUI):** main menu → host / join / vs bot / score-attack;

lobby with class + magic-slot pick; post-match rematch prompt.

Moment-to-moment texture the player reads: hit-stop on heavy connects, Wwise-driven hit reactions, and rollback-clean cancels — an interrupted ability never leaves a sound or effect behind (existing engine guarantee).

---

4. AI / Agent Architecture

Two layers: a **dev-time crew** that builds game content through the Claude + Unreal Engine MCP pipeline, and **one runtime agent** whose calls are confined to the round break so the fight itself never waits on a model. Bots are deterministic C++ state machines — the LLM tunes them between rounds; it never steers them mid-round (multiplayer correctness rule: nothing non-deterministic in the simulation).

4.1 Dev-Time Crew (Claude + Unreal MCP, in-editor)

Agent	Role (one sentence)	Output format
Arena Architect	Designs and blocks out each arena in-editor via the UE MCP — geometry, spawn points, sightlines — from a one-paragraph brief.	Editor edits + `arena_manifest.json` (bounds, spawns, hazard list)
Bot Trainer	Generates and tunes bot behavior parameters per difficulty tier against the combat triangle.	`DT_BotTuning` DataTable rows (aggression, parry %, reaction ms, combo depth)
Combat QA	Runs automated matches (bot vs bot), reads logs and screenshots, and files reproducible defect reports.	`qa_report.json` (defect, repro steps, severity)
Balance Analyst	Reads match telemetry and proposes tuning diffs when a class or option exceeds win-rate bounds (55% triggers review).	DataTable diff + one-paragraph rationale

Gameplay effect of the crew: the arenas, the bots, and the balance the player experiences are agent-produced, human-approved. Every crew output is data (JSON/DataTables) reviewed before merge — agents never commit directly.

4.2 Runtime Agent (in the shipped game)

Ringside Agent — the only model call in the running game.

- **Role sentence:** At each round break, generates the announcer line, one

coach tip from the round's telemetry, and a bot-persona adjustment for the next round.

- **Gameplay effect:** rounds feel watched and narrated; solo players get

coaching that names their actual habit ("you always dash left after blocking"); bots feel adaptive between rounds without ever being non-deterministic within one.

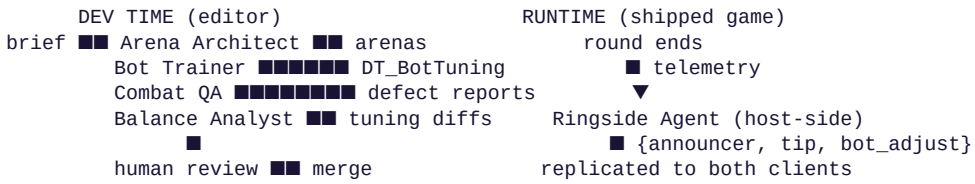
- **Output format:** `{announcer_line (≤ 20 words), coach_tip (≤ 25 words,

references ≥ 1 telemetry stat), bot_adjust {aggression_delta, style ∈ {rushdown, turtle, spacing}} | null}`.

- **Multiplayer correctness:** the host (server) makes the call at round

end, replicates the resulting strings to both clients; the round-break countdown does not wait — if the reply misses the 3-second window, canned lines ship instead. The match is never blocked by the API.

4.3 Interaction Diagram



---

5. Technical Strategy

5.1 Stack (existing foundation in bold — already shipped and stable)

- **UE 5.6, pure native C++, no-Tick event-driven architecture**
- **GAS combat core: PlayerState-owned ASC, input buffer, prediction keys**
- **Soft-lock melee (`SoftLockTarget` replicated source of truth)**
- **Steam OSS sessions (`OSSessionsSubsystem`), Wwise, Perforce**
- New this project: round/match flow (`OSRoundSubsystem`), bot state

machines, CommonUI front end, Ringside Agent HTTP client (`FHttpModule`, host-side only)

5.2 Model Assignment and Token Budget

Dev-time crew runs on Claude Code (Sonnet-class) under the existing 20-skill UE5 library and `.mdc` rules — development cost, not game cost.

Runtime budget (the shipped game):

Call	Model	Calls / match	Tokens in	Tokens out	Match total
Ringside Agent	Claude Haiku	≤ 5 (one per round break)	900 (telemetry + persona)	120	≈ 5,100

≈ **$0.006 per match** — effectively free. Hard cap: 10 calls/match; on cap, timeout (> 3 s), or API error, the game falls back to canned announcer/tip lines. Players without connectivity lose flavor, never function.

5.3 Constraints (named, with the design decision each caused)

1. **Constraint: multiplayer determinism.** An LLM cannot be in the

simulation path of a predicted, server-authoritative fighting game. *Therefore* the Ringside Agent acts only between rounds, host-side, with replicated string output and a canned-line fallback — bots

remain deterministic C++ within a round.

2. **Constraint: five weeks, one principal.** New engineering surface must stay near zero. *Therefore* the game is built entirely on shipped systems (GAS core, soft-lock, sessions); the only new gameplay code is round flow and bot state machines — both junior-executable patterns.
3. **Constraint: small-PvP population risk.** An indie PvP game with an empty queue is dead at week two. *Therefore* bots and score-attack ship at launch, and the online mode is invite-first (listen server) rather than quickmatch-dependent.

5.4 Latency Plan

Player-visible round break is a fixed 5-second countdown; the Ringside call budget is 3 seconds inside it. In-round combat uses the existing prediction/rollback stack — no model calls, no new latency surface.

6. Scope & Development Plan

Shipped scope: 1v1 online (host + invite) and vs bot; 2 classes (blade, caster), 1 swappable magic slot each; best-of-5 rounds; 2 arenas; bots at 3 tiers; score-attack mode; CommonUI front end (menu, lobby, HUD); Ringside Agent with canned fallback; Steam demo build. **Nothing else.**

Explicitly cut: 2v2, third class (hybrid), quickmatch, Domination, GameLift, ranked, cosmetics, achievements. (2v2 and class three are the studio's fast-follows, not capstone scope.)

Week by week (each week ends runnable):

1. **W1 — Fun proof:** Arena Architect blocks out arena 1 via UE MCP; round flow (`OSRoundSubsystem`); 1v1 vs dumb bot, best-of-5, local. *Gate: "one more round" felt in internal play.*
2. **W2 — Online:** Steam listen server host/invite path through `OSSessionsSubsystem`; class 2 loadout; bot tiers 1–2 via Bot Trainer.
3. **W3 — Full loop:** Ringside Agent + telemetry; score-attack mode; arena 2; bot tier 3; Combat QA agent running nightly bot-vs-bot matches.
4. **W4 — Balance & front end:** Balance Analyst pass on QA telemetry; CommonUI menu/lobby/rematch flow; Wwise polish on round transitions.
5. **W5 — Ship:** Steam build + demo depot, playtest, capstone presentation. Buffer for W1–4 slippage.

First cut if behind: score-attack mode (the bots and arenas it reuses survive; only the gauntlet wrapper is dropped).

---

7. Success Criteria

- A stranger can install the Steam demo, fight a bot, and host a friend

1v1 without instructions.

- Playtesters ask for "one more round" unprompted (the pitch doc's gate

metric).

- Every arena, bot tier, and balance pass in the build traces to a named

agent output that was human-reviewed — the pipeline is demonstrable, not anecdotal.

---

8. Revision Note (Assignment #1 → Final Draft)

What changed: the capstone game. Assignment #1 specified *Last Call*, a browser-based AI interrogation game. The final draft replaces it with

Slash Roller: Duels, the scoped-down cut of the UE5 game my studio team is actively building.

Why — this is the Class 03 method applied honestly: stress-testing the first draft surfaced that its strongest quality (a self-contained LLM game) was also its flaw as *my* capstone: it exercised none of my existing engineering, and its five weeks of work would be abandoned after the course. Meanwhile the studio project already had the scope problem Class 03 warns about (a 122-item Early Access list — documented scope creep). Applying the review discipline to the real project produced this cut: reuse the shipped combat foundation, keep the multi-agent pipeline as both dev method and runtime feature, and ship on Steam.

Meaningful changes visible in this draft (rubric: Revision & Growth):

1. **Scope creep addressed by deletion:** the studio vision's mode list,

class roster, and dedicated-server plan are cut to a 6-item shipped-scope list ending in "Nothing else."

2. **Agent roles re-specified by output format:** the first draft's four

runtime agents are replaced by four dev-time agents + one runtime agent, each with a named data deliverable (DataTables, JSON manifests) instead of free-text behavior.

3. **The LLM moved out of the hot path:** the first draft ran three model

calls per player action; this draft runs at most one per round break with a canned fallback — the multiplayer-correct redesign of the same idea.